

LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 11
EXCEPTION



Disusun Oleh:
M Rizki Ruswan 105224018

PROGRAM STUDI ILMU KOMPUTER
FAKULTAS SAINS DAN KOMPUTER
UNIVERSITAS PERTAMINA
2026

I. Pendahuluan

Pada tugas ini saya membuat sebuah program Sistem Reservasi Tiket Kereta Api bernama JAVA EXPRESS menggunakan bahasa pemrograman Java. Program ini dirancang untuk mensimulasikan proses pemesanan tiket kereta api melalui Command Line Interface (CLI) dengan menerapkan konsep Object-Oriented Programming (OOP) dan Exception Handling.

Sistem menyediakan beberapa fitur utama, yaitu melihat jadwal kereta, melakukan pemesanan tiket, serta keluar dari aplikasi. Data kereta telah tersedia secara otomatis saat program dijalankan sehingga pengguna dapat langsung melakukan reservasi.

Dalam pengembangannya, saya membuat beberapa class yang memiliki tugas masing-masing. Class KeretaApi digunakan sebagai model data kereta, class ReservasiTiket berfungsi sebagai pusat pengelolaan reservasi, sedangkan class Main digunakan untuk menjalankan seluruh sistem.

Selain itu, saya juga menerapkan konsep Checked Exception dan Unchecked Exception untuk menangani berbagai kemungkinan kesalahan input pengguna. Dengan adanya mekanisme exception handling, program menjadi lebih aman, stabil, dan tidak mudah mengalami crash ketika menerima input yang tidak sesuai.

II. Variabel

No	Nama Variabel	Tipe Data	Fungsi
1	kode	String	Menyimpan kode unik kereta api sebagai identitas setiap kereta.
2	nama	String	Menyimpan nama kereta api.
3	rute	String	Menyimpan rute perjalanan kereta api.
4	sisaKursi	int	Menyimpan jumlah kursi yang masih tersedia.
5	daftarKereta	ArrayList	Digunakan untuk menyimpan seluruh data kereta yang tersedia dalam sistem.
6	kodeKereta	String	Menyimpan kode kereta yang dipilih pengguna saat melakukan pemesanan.
7	nik	String	Menyimpan NIK penumpang yang melakukan reservasi.
8	namaPenumpang	String	Menyimpan nama penumpang yang melakukan reservasi.

9	jumlahTiket	int	Menyimpan jumlah tiket yang ingin dipesan oleh pengguna.
10	keretaDitemukan	KeretaApi	Menyimpan objek kereta hasil pencarian berdasarkan kode kereta.
11	pilihan	int	Menyimpan pilihan menu yang dipilih pengguna pada program utama.
12	reservasi	ReservasiTiket	Digunakan untuk membuat objek ReservasiTiket dan menjalankan seluruh proses reservasi.
13	input	Scanner	Digunakan untuk menerima input dari pengguna melalui keyboard.

III. Constructor dan Method

No	Nama Method	Jenis Method	Fungsi
1	KeretaApi(String kode, String nama, String rute, int sisaKursi)	Constructor	Membuat objek kereta api baru dengan kode, nama, rute, dan jumlah kursi yang tersedia.
2	ReservasiTiket()	Constructor	Menginisialisasi data kereta api yang tersedia pada sistem reservasi.
3	getKode()	Getter	Mengambil nilai kode kereta api.
4	getNama()	Getter	Mengambil nama kereta api.
5	getRute()	Getter	Mengambil rute perjalanan kereta api.
6	getSisaKursi()	Getter	Mengambil jumlah kursi yang masih tersedia.
7	kurangiKursi(int jumlah)	Procedural	Mengurangi jumlah kursi yang tersedia setelah pemesanan berhasil dilakukan.

8	tampilkanJadwal()	Procedural	Menampilkan seluruh jadwal dan informasi kereta yang tersedia pada sistem.
9	pesanTiket(String kodeKereta, String nik, String nama, int jumlahTiket)	Procedural	Memproses pemesanan tiket dengan melakukan validasi data penumpang, pencarian kereta, dan pengecekan ketersediaan kursi.
10	DataPenumpangTidakValidException(String pesan)	Constructor	Membuat exception ketika data NIK penumpang tidak valid.
11	RuteTidakDitemukanException(String pesan)	Constructor	Membuat exception ketika kode atau rute kereta tidak ditemukan.
12	TiketHabisException(String namaKereta, int sisaKursi)	Constructor	Membuat exception ketika jumlah tiket yang diminta melebihi jumlah kursi yang tersedia.
13	getNamaKereta()	Getter	Mengambil nama kereta pada exception TiketHabisException.
14	getSisaKursi()	Getter	Mengambil informasi sisa kursi pada exception TiketHabisException.
15	main(String[] args)	Static Method	Menjalankan seluruh sistem JAVA EXPRESS serta menangani interaksi pengguna dan exception yang terjadi.

IV. Dokumentasi dan Pembahasan Code

1.

```
THT > src > KeretaApi.java > KeretaApi
1  public class KeretaApi {
2      private String kode;
3      private String nama;
4      private String rute;
5      private int sisaKursi;
6
7      public KeretaApi(String kode, String nama, String rute, int sisaKursi) {
8          this.kode = kode;
9          this.nama = nama;
10         this.rute = rute;
11         this.sisaKursi = sisaKursi;
12     }
13
14     public String getKode() {
15         return kode;
16     }
17
18     public String getNama() {
19         return nama;
20     }
21
22     public String getRute() {
23         return rute;
24     }
25
26     public int getSisaKursi() {
27         return sisaKursi;
28     }
29
30     public void kurangiKursi(int jumlah) {
31         sisaKursi -= jumlah;
32     }
33
34     @Override
35     public String toString() {
36         return kode + " | " + nama + " | " + rute + " | Sisa Kursi : " + sisaKursi;
37     }
38 }
```

Penjelasan:

Pada bagian ini saya membuat class `KeretaApi` yang berfungsi untuk menyimpan data kereta api dalam sistem reservasi. Class ini memiliki empat atribut yaitu `kode`, `nama`, `rute`, dan `sisaKursi` yang digunakan untuk menyimpan informasi setiap kereta.

Saya juga membuat constructor `KeretaApi()` yang digunakan untuk menginisialisasi data kereta saat objek dibuat. Selanjutnya, saya membuat beberapa method getter yaitu `getKode()`, `getNama()`, `getRute()`, dan `getSisaKursi()` untuk mengambil nilai dari setiap atribut.

Selain itu, terdapat method kurangiKursi() yang berfungsi mengurangi jumlah kursi yang tersedia setelah proses pemesanan tiket berhasil dilakukan. Terakhir, saya melakukan override pada method toString() agar informasi kereta dapat ditampilkan dalam bentuk teks yang lebih mudah dibaca.

Dengan adanya class KeretaApi, data setiap kereta dapat dikelola dengan lebih terstruktur dan menjadi dasar dalam proses reservasi tiket pada sistem JAVA EXPRESS.

2.

```
THT > src > DataPenumpangTidakValidException.java > DataPenumpangTidakValidException
1 public class DataPenumpangTidakValidException extends RuntimeException {
2
3     public DataPenumpangTidakValidException(String pesan) {
4         super(pesan);
5     }
6 }
```

Penjelasan:

Pada bagian ini saya membuat class DataPenumpangTidakValidException yang merupakan custom exception dengan mewarisi (extends) class RuntimeException. Exception ini digunakan untuk menangani kesalahan ketika data penumpang yang dimasukkan tidak sesuai dengan ketentuan yang telah ditetapkan oleh sistem.

Class ini memiliki constructor DataPenumpangTidakValidException(String pesan) yang menerima parameter berupa pesan kesalahan. Kemudian pesan tersebut dikirim ke class induk menggunakan super(pesan) sehingga dapat ditampilkan saat exception terjadi.

Dengan adanya class ini, saya dapat memberikan informasi kesalahan yang lebih spesifik kepada pengguna, misalnya ketika NIK kosong, panjang NIK tidak sesuai, atau data penumpang yang dimasukkan tidak valid. Hal ini membuat proses validasi data menjadi lebih terstruktur dan mudah dipahami.

3.

```
THT > src > RuteTidakDitemukanException.java > RuteTidakDitemukanException
1 public class RuteTidakDitemukanException extends Exception {
2
3     public RuteTidakDitemukanException(String pesan) {
4         super(pesan);
5     }
6 }
```

Penjelasan:

Pada bagian ini saya membuat class `RuteTidakDitemukanException` yang merupakan custom exception dengan mewarisi (extends) class `Exception`. Exception ini digunakan untuk menangani kondisi ketika kode atau rute kereta yang dimasukkan pengguna tidak ditemukan dalam daftar kereta yang tersedia pada sistem.

Class ini memiliki constructor `RuteTidakDitemukanException(String pesan)` yang menerima parameter berupa pesan kesalahan. Pesan tersebut kemudian diteruskan ke class induk menggunakan `super(pesan)` sehingga dapat ditampilkan saat exception terjadi.

Dengan adanya class ini, saya dapat memberikan informasi kesalahan yang lebih jelas kepada pengguna ketika memilih kereta yang tidak tersedia. Selain itu, penggunaan custom exception membuat proses penanganan error menjadi lebih terstruktur dan memudahkan proses debugging pada sistem reservasi tiket kereta api.

4.

```
THT > src >  TiketHabisException.java >  TiketHabisException
1  public class TiketHabisException extends Exception {
2
3      private String namaKereta;
4      private int sisaKursi;
5
6      public TiketHabisException(String namaKereta, int sisaKursi) {
7          super(message: "Tiket tidak mencukupi");
8
9          this.namaKereta = namaKereta;
10         this.sisaKursi = sisaKursi;
11     }
12
13     public String getNamaKereta() {
14         return namaKereta;
15     }
16
17     public int getSisaKursi() {
18         return sisaKursi;
19     }
20 }
```

Penjelasan:

Pada bagian ini saya membuat class `TiketHabisException` yang merupakan custom exception dengan mewarisi (extends) class `Exception`. Exception ini digunakan untuk menangani kondisi ketika jumlah tiket yang diminta oleh pengguna melebihi jumlah kursi yang masih tersedia pada kereta yang dipilih.

Class ini memiliki dua atribut, yaitu `namaKereta` yang bertipe `String` dan `sisaKursi` yang bertipe `int`. Kedua atribut tersebut digunakan untuk menyimpan

informasi kereta yang mengalami kekurangan tiket sehingga pesan kesalahan dapat ditampilkan dengan lebih detail.

Selanjutnya, saya membuat constructor `TiketHabisException(String namaKereta, int sisaKursi)` yang menerima nama kereta dan jumlah kursi yang tersisa. Pada constructor ini saya menggunakan `super("Tiket tidak mencukupi")` untuk mengirimkan pesan kesalahan ke class induk, kemudian menyimpan nilai parameter ke dalam atribut menggunakan kata kunci `this`.

Selain itu, saya membuat method `getNamaKereta()` dan `getSisaKursi()` yang berfungsi untuk mengambil informasi nama kereta dan jumlah kursi yang masih tersedia ketika exception ditangani.

Dengan adanya class `TiketHabisException`, saya dapat memberikan informasi yang lebih lengkap kepada pengguna mengenai kereta yang dipilih dan jumlah kursi yang tersisa, sehingga proses penanganan kesalahan menjadi lebih informatif dan mudah dipahami.

5.

```
THT > src > ReservasiTiket.java > ReservasiTiket
1  import java.util.ArrayList;
2
3  public class ReservasiTiket {
4      private ArrayList<KeretaApi> daftarKereta;
5      public ReservasiTiket() {
6          daftarKereta = new ArrayList<>();
7          daftarKereta.add( new KeretaApi( kode: "K01", nama: "Argo Bromo", rute: "JKT - SBY", sisaKursi: 50));
8          daftarKereta.add( new KeretaApi( kode: "K02", nama: "Parahyangan", rute: "JKT - BDG", sisaKursi: 15));
9      }
10     public void tampilkanJadwal() {
11         System.out.println(x: "\n===== JADWAL KERETA =====");
12         for (KeretaApi k : daftarKereta) { System.out.println(k);
13     }
14 }
15 public void pesanTiket(String kodeKereta, String nik, String nama, int jumlahTiket) throws RuteTidakDitemukanException,
16     TiketHabisException {
17     if (nik.length() != 16) {
18         throw new DataPenumpangTidakValidException( pesan: "NIK harus 16 digit");
19     }
20     if (!nik.matches(regex: "\\d+")) {
21         throw new DataPenumpangTidakValidException( pesan: "NIK hanya boleh angka");
22     }
23     KeretaApi keretaDitemukan = null;
24     for (KeretaApi k : daftarKereta) {
25         if (k.getKode().equalsIgnoreCase(kodeKereta)) {
26             keretaDitemukan = k;
27             break;
28         }
29     }
30     if (keretaDitemukan == null) {
31         throw new RuteTidakDitemukanException( pesan: "Kode kereta tidak ditemukan");
32     }
33     if (jumlahTiket >
34         keretaDitemukan.getSisaKursi()) {
35         throw new TiketHabisException( keretaDitemukan.getNama(), keretaDitemukan.getSisaKursi());
36     }
37     keretaDitemukan.kurangiKursi(jumlahTiket);
38     System.out.println(x: "\n=== PEMESANAN BERHASIL ===");
39     System.out.println("Nama : " + nama);
40     System.out.println("NIK : " + nik);
41     System.out.println("Kereta : " + keretaDitemukan.getNama());
42     System.out.println("Jumlah Tiket : " + jumlahTiket);
43 }
44 }
45 }
```

Penjelasan:

Pada bagian ini saya membuat class `ReservasiTiket` yang berfungsi untuk mengelola seluruh proses reservasi tiket kereta api. Class ini menyimpan daftar kereta yang tersedia menggunakan `ArrayList<KeretaApi>` sehingga data kereta dapat dikelola secara dinamis.

Pada constructor `ReservasiTiket()`, saya menginisialisasi objek `ArrayList` dan menambahkan dua data kereta yaitu Argo Bromo dan Parahyangan sebagai data awal yang dapat dipilih oleh pengguna.

Selanjutnya, saya membuat method `tampilkanJadwal()` yang berfungsi untuk menampilkan seluruh daftar kereta yang tersedia. Method ini menggunakan perulangan `for-each` untuk menampilkan setiap objek kereta yang terdapat dalam `ArrayList`.

Method utama pada class ini adalah `pesanTiket()` yang digunakan untuk memproses pemesanan tiket. Pada method ini saya melakukan beberapa validasi, yaitu memastikan NIK terdiri dari 16 digit dan hanya berisi angka. Jika data tidak valid, sistem akan melempar `DataPenumpangTidakValidException`.

Setelah validasi NIK berhasil, saya melakukan pencarian kereta berdasarkan kode yang dimasukkan pengguna. Jika kode kereta tidak ditemukan, sistem akan melempar `RuteTidakDitemukanException`. Selanjutnya, sistem akan memeriksa apakah jumlah tiket yang diminta melebihi jumlah kursi yang tersedia. Jika kursi tidak mencukupi, sistem akan melempar `TiketHabisException`.

Apabila seluruh proses validasi berhasil, sistem akan mengurangi jumlah kursi yang tersedia menggunakan method `kurangiKursi()`, kemudian menampilkan informasi pemesanan yang berhasil dilakukan oleh pengguna.

Dengan adanya class `ReservasiTiket`, saya dapat mengelola proses reservasi, validasi data, pencarian kereta, dan penanganan kesalahan secara terstruktur dalam satu class.

6.

```
THT > src > Main.java > Main > main(String[])
1  import java.util.InputMismatchException;
2  import java.util.Scanner;
3
4  public class Main {
5
6      Run | Debug
7      public static void main(String[] args) {
8          Scanner input = new Scanner(System.in);
9          ReservasiTiket reservasi = new ReservasiTiket();
10         int pilihan = 0;
11         do {
12             try {
13                 System.out.println(x: "\n=====");
14                 System.out.println(x: "    JAVA EXPRESS");
15                 System.out.println(x: "=====");
16                 System.out.println(x: "1. Lihat Jadwal");
17                 System.out.println(x: "2. Pesan Tiket");
18                 System.out.println(x: "3. Keluar");
19                 System.out.print(s: "Pilih Menu : ");
20
21                 pilihan = input.nextInt();
22                 input.nextLine();
23                 switch (pilihan) {
24                     case 1:
25                         reservasi.tampilkanJadwal();
26                         break;
27                     case 2:
28                         System.out.print( s: "Kode Kereta : ");
29                         String kode = input.nextLine();
30                         System.out.print(s: "NIK : ");
31                         String nik = input.nextLine();
32                         System.out.print(s: "Nama : ");
33                         String nama = input.nextLine();
34                         System.out.print(s: "Jumlah Tiket : ");
35                         int jumlah = input.nextInt();
36                         input.nextLine();
37                         reservasi.pesanTiket(kode, nik, nama, jumlah);
38                         break;
39                     case 3:
40                         System.out.println( x: "Terima kasih telah menggunakan JAVA EXPRESS");
41                         break;
42                     default:
43                         System.out.println(x: "Menu tidak tersedia");
44                 }
45             } catch (InputMismatchException e) {
46                 System.out.println( x: "Input harus berupa angka!");
47                 input.nextLine();
48             }
49             catch (DataPenumpangTidakValidException e) {
50                 System.out.println("Error Data Penumpang : " + e.getMessage());
51             }
52             catch (RuteTidakDitemukanException e) {
53                 System.out.println("Error : " + e.getMessage());
54             }
55             catch (TiketHabisException e) {
56                 System.out.println( x: "Tiket tidak mencukupi!");
57                 System.out.println("Kereta : " + e.getNamaKereta());
58                 System.out.println("Sisa Kursi : " + e.getSisaKursi());
59             }
60             finally {
61                 System.out.println(x: "\n[ Sistem siap menerima input berikutnya ]");
62             }
63         } while (pilihan != 3);
64         input.close();
65     }
66 }
```

Penjelasan:

Pada bagian ini saya membuat class Main yang berfungsi sebagai titik awal eksekusi program sistem reservasi tiket kereta api JAVA EXPRESS. Di dalam method main(), saya membuat objek Scanner untuk menerima input dari pengguna dan objek ReservasiTiket untuk menjalankan seluruh fitur reservasi tiket.

Selanjutnya, saya menggunakan perulangan do-while untuk menampilkan menu utama secara berulang hingga pengguna memilih menu keluar. Menu yang tersedia yaitu melihat jadwal kereta, memesan tiket, dan keluar dari program.

Pada menu Lihat Jadwal, sistem akan memanggil method `tampilkanJadwal()` untuk menampilkan daftar kereta yang tersedia. Sedangkan pada menu Pesan Tiket, pengguna diminta memasukkan kode kereta, NIK, nama, dan jumlah tiket yang ingin dipesan. Data tersebut kemudian diproses menggunakan method `pesanTiket()`.

Saya juga menerapkan beberapa exception handling menggunakan blok try-catch untuk menangani berbagai kemungkinan kesalahan. `InputMismatchException` digunakan ketika pengguna memasukkan data yang tidak sesuai tipe yang diharapkan. `DataPenumpangTidakValidException` digunakan untuk menangani kesalahan pada data penumpang, `RuteTidakDitemukanException` digunakan ketika kode kereta tidak ditemukan, dan `TiketHabisException` digunakan ketika jumlah tiket yang diminta melebihi kursi yang tersedia.

Selain itu, saya menggunakan blok finally yang akan selalu dijalankan setelah proses selesai, baik terjadi error maupun tidak. Pada bagian ini sistem menampilkan pesan bahwa program siap menerima input berikutnya.

Dengan adanya class `Main`, saya dapat mengatur alur jalannya program, menerima input pengguna, menjalankan fitur reservasi, serta menangani berbagai kondisi kesalahan agar program dapat berjalan dengan lebih aman dan terstruktur.

V. Kesimpulan

Berdasarkan hasil implementasi dan pengujian program Sistem Reservasi Tiket Kereta Api JAVA EXPRESS, saya berhasil menerapkan konsep Object-Oriented Programming (OOP) menggunakan beberapa class yang saling berhubungan, yaitu *KeretaApi*, *ReservasiTiket*, dan class exception yang dibuat secara khusus. Program ini mampu menampilkan jadwal kereta, memproses pemesanan tiket, serta mengelola data kursi yang tersedia.

Selain itu, saya juga berhasil menerapkan konsep Exception Handling untuk menangani berbagai kondisi kesalahan, seperti data penumpang yang tidak valid, kode kereta yang tidak ditemukan, dan jumlah tiket yang melebihi kapasitas kursi yang tersedia. Dengan adanya penanganan exception tersebut, program menjadi lebih aman dan mampu memberikan informasi kesalahan yang jelas kepada pengguna.

Melalui praktikum ini, saya memperoleh pemahaman yang lebih baik mengenai penerapan OOP, penggunaan custom exception, validasi data, serta pengelolaan alur program menggunakan mekanisme try-catch-finally dalam bahasa pemrograman Java.

IV. Daftar Pustaka

Oracle. (2023). *Java Documentation*. <https://docs.oracle.com/en/java/>